> Tracker ID: FN-26 · Generated: 2026-10-14 · Source: Komol-এর উত্তর (২০২৬-১০-১৪), `_memory/*`, Sokrio কোডবেস + git history, FN SE job post Decision: D9 (`_memory/decisions.md`) · Q-bank: FN-27 (`03-roles/fieldnation-software-engineer/04-behavioral/04-hm-technical-depth-qbank.md`)

Field Nation — Hiring Manager / Tech Team রাউন্ড: ৮-দিনের "Technical Depth" প্ল্যান

০. এই রাউন্ডটা কী, আর কী না

	Technical রাউন্ড (আগেরটা)	Hiring Manager / Tech Team রাউন্ড (এটা)
প্রশ্ন	"Event loop কী?" "এই প্রবলেম কোড করো"	"Walk me through something you built. **Why** that way? What broke? What would you change?"
মাপে	সঠিক উত্তর জানো কি না	**গভীরতা, মালিকানা (I vs we), trade-off বিচার, সততা, কমিউনিকেশন**
কে	Engineer	**US-side Engineering Manager + ১–২ জন engineer (panel)**, ইংরেজিতে ভিডিও কল
কীভাবে খোঁড়ে	চওড়া (অনেক টপিক)	**লম্বা (একটা প্রজেক্টে ৪–৫ লেভেল নিচে)**, প্রতিবার "why?" জিজ্ঞেস করে

Depth Ladder: প্রতিটা প্রজেক্টে ইন্টারভিউয়ার এই সিঁড়ি বেয়ে নামবে:

```
L1 WHAT     — কী বানিয়েছিলে? (৩০ সেকেন্ড, বড় ছবি)
L2 HOW      — ভেতরে কীভাবে কাজ করে? (ডেটা ফ্লো, কম্পোনেন্ট)
L3 WHY      — কেন এভাবে? কী কী বিকল্প ছিল, কেন বাদ দিলে?
L4 BREAK    — কোথায় ভাঙে? স্কেল ১০x হলে? কী ভুল হয়েছিল?
L5 REFLECT  — আজ আবার করলে কী বদলাতে? তোমার নিজের অংশ ঠিক কোনটা?
```

> যেকোনো প্রজেক্টে **L4–L5 পর্যন্ত** কনফিডেন্টলি নামতে পারলে এই রাউন্ড জেতা যায়। বেশিরভাগ ক্যান্ডিডেট L2-তে আটকে যায়।

US ম্যানেজারের সাথে কমিউনিকেশন নিয়ম:

1. **BLUF (Bottom Line Up Front):** আগে উত্তর, পরে ব্যাখ্যা। "The short answer is X. Here's why..."
2. **"I" vs "we" পরিষ্কার রাখো:** "The team owned X; **I** designed Y and wrote Z."
3. **সংখ্যার সাথে সবসময় পদ্ধতি বলো:** "~85% faster, measured as p50 load time on the `<report>`, `<X>s → <Y>s`" (আসল সংখ্যা Receipt Card থেকে)। পদ্ধতি না জানলে সংখ্যা বলবে না।
4. **"I don't know" + reasoning** বানানো উত্তরের চেয়ে অনেক ভালো: "I haven't run that in production. Here's how I'd reason about it..."
5. প্রতিটা উত্তর **৯০ সেকেন্ডের মধ্যে**, তারপর থামো: "Want me to go deeper on any part?"

---

১. 🔴 সবার আগে: Truth Audit (কোডবেস থেকে পাওয়া ঝুঁকি)

২০২৬-১০-১৪-এ লোকাল Sokrio রিপো (`sokrio-server`, দুই frontend, mobile, benchmark-python, benchmark) স্ক্যান করে পাওয়া গেছে:

CV-র দাবি	কোডে কী পাওয়া গেল	ঝুঁকি	কী করবে
Monolith → microservices, **RabbitMQ** দিয়ে	কোনো রিপোতে `rabbitmq` / `amqp` নেই। Event bus = **Kafka** (`app/Contracts/Kafka/*`, `KafkaServiceProvider`, `PublishesKafkaMessage` observer)	🔴 খুব বেশি: "RabbitMQ exchange type কী ছিল?" জিজ্ঞেস করলেই ধরা পড়বে	RabbitMQ কি অন্য রিপোতে (Notification/SOP service) আছে, যা লোকালে নেই? **থাকলে** রিপোর নাম আর তোমার অংশ লেখো। **না থাকলে** উত্তর আর CV-র লাইন বদলাও: "event-driven over a message broker (Kafka)"
Kafka: "basics"	Kafka producer/consumer, consumer-group isolation fix, dynamic group ID + retry, consumer max runtime 6h। **git author: টিমমেট (Rakib), ২০২৫-১০ → ১২**	🟡 মাঝারি	তোমার সম্পর্ক কী: review করেছ? ব্যবহার করেছ? on-call-এ debug করেছ? `[Komol পূরণ করবে]`। সৎ ফ্রেম: "A teammate built our Kafka layer; I work with it daily, e.g. ..."
Node.js / Express services	কোনো রিপোতে Node backend নেই (`express` / `@nestjs/core` / `fastify` নেই)	🔴 বেশি: FN Node-এর দিকে যাচ্ছে, তাই অবশ্যই জিজ্ঞেস করবে	Node কাজটা কোথায় হয়েছিল (SoftArch? Am Coders? সাইড প্রজেক্ট?) `[Komol পূরণ করবে]`। প্রমাণ বানাতে `fn-lite` NestJS রিপো
২০%, ৬৫%, ৮৫%, ২৭%, ৯৯.৯%	মাপার পদ্ধতি কোথাও লেখা নেই (handoff-এও খোলা ঝুঁকি)	🔴 বেশি	প্রতিটার জন্য Receipt Card (§৩, Day 1)
Laravel version	কোড: **Laravel 8** (`laravel/framework: ^8.0`)	🟢 কম	"Laravel 10" বলবে না

✅ কোড থেকে পাওয়া নতুন, যাচাইযোগ্য তথ্য (নিরাপদে বলা যায়):

- `sokrio-server`-এ ১১,৫৬১ commit-এর মধ্যে **১,৬৬১টা তোমার (~১৪%)**, ২০২৩-০৮-২৮ থেকে
- Multi-tenant: subdomain → `EnsureTenantAccess` → `TenantManager::reconnect()` একসাথে MySQL **আর** MongoDB সুইচ করে (একই db নাম)। Org lookup ৭ দিন cache করা থাকে। Read-replica connection আছে।
- **গভীর gotcha (L4-এর সোনা):** `ScopeCacheToTenant` শুধু HTTP request-এ cache prefix দেয়, queue job-এ দেয় না। তাই job request-এর লেখা key দেখতে পায় না।
- Market-benchmark optimization: all-tenant fan-out (`gather_tenants`)-কে root cause হিসেবে ধরা, adaptive concurrency `[8, 24]` (~প্রতি ৫ টেন্যান্টে ১ স্লট), TTL 600s → 10800s (**১৮× কম recompute**), lookback ৩ মাস → ডিফল্ট ১, index-check স্ক্রিপ্ট
- Monthly rollups (`order_rollups_monthly`, per-tenant), scheduled job, event-driven না। **কেন:** `order_summaries` লেখে Laravel, তাই event hook লাগলে অন্য সার্ভিসে কাপলিং বাড়ত।
- Structured JSON-lines logging: `{tenant, level, duration_ms, cache_hit, error_code, request_id}`
- Task Manager ডিজাইন: MongoDB-র বদলে MySQL (relational join), নতুন `user_supervisors` টেবিল (audit করে নিশ্চিত হয়ে), one-way `achievement_locked_at`

- QA Test Lab (২০২৬-১০): write-ahead journal (pending → applied, পুরো `before` row সহ), reverse-order revert, local-only write guard
- **একটা আসল revert:** `ReportTooLargeException` + stock-details report পরিবর্তন (২০২৬-০৮-১৩), দুই দিন পর revert (০৮-১৫)। এটা S11 (mistake) স্টোরি হতে পারে, যদি তোমার মনে থাকে কেন revert করা হয়েছিল।

---

২. সময়সূচি (তারিখ ঠিক না, তাই D-8 → D-0)

ডিফল্ট শুরু: **বৃহঃ ২০২৬-১০-১৫ = D-8**। তারিখ জানলে D-0 বসিয়ে পেছনে গুনে নাও। সপ্তাহের দিনে ~৩–৪ ঘণ্টা, শুক্র/শনিতে ~৬ ঘণ্টা।

দিন	ডিফল্ট তারিখ	ফোকাস	আউটপুট (শেষে কী থাকবে)
D-8	বৃহঃ ১০-১৫	🔴 Truth Audit + Receipt Cards	§১-এর প্রতিটা 🔴 সারি resolved; ৬টা Receipt Card
D-7	শুক্র ১০-১৬ (৬ ঘণ্টা)	Anchor Projects A1–A3 → Depth Ladder	৩টা প্রজেক্ট L1–L5 লেখা + জোরে বলা (রেকর্ড)
D-6	শনি ১০-১৭ (৬ ঘণ্টা)	Anchor Projects A4–A6 + আর্কিটেকচার আঁকা	৩টা প্রজেক্ট L1–L5 + Sokrio-র ১-পাতার ডায়াগ্রাম মুখস্থ
D-5	রবি ১০-১৮	Architecture "Why X not Y" কার্ড	Q-bank §B-এর ১০টা কার্ড, প্রতিটা ৯০ সেকেন্ডে
D-4	সোম ১০-১৯	Incidents, Debugging, Ownership	S4 incident + S11 mistake + S10 disagreement পুরো ভরাট; debugging walkthrough
D-3	মঙ্গল ১০-২০	Gaps: Node / NestJS / K8s / Kafka + Field Nation-এর জায়গায় নিজেকে বসানো	"Honest Bridge" উত্তর ×৪; FN-এর সমস্যায় Sokrio অভিজ্ঞতা ম্যাপ করা
D-2	বুধ ১০-২১	🎤 Live Mock HM-1 (৬০ মিনিট, panel স্টাইল)	Fix list → `weak-areas.md`
D-1	বৃহঃ ১০-২২	Fix list + Mock HM-2 (৩০ মিনিট, শুধু L4–L5 প্রশ্ন) + ম্যানেজারকে করার প্রশ্ন	সব fix বন্ধ; ৫টা প্রশ্ন রেডি
D-0	ইন্টারভিউর দিন	হালকা: Receipt Cards + pitch একবার, সেটআপ চেক	—

প্রতিদিনের অভ্যাস (~৩০ মিনিট): ১টা anchor project জোরে বলা (২ মিনিট L1–L2, তারপর নিজেকে "why?" ৩ বার) · ১টা DSA (TypeScript) যাতে হাত না জমে · দিনের শেষে `handoff.md` আপডেট।

---

৩. দিন-ভিত্তিক বিস্তারিত

D-8 — Truth Audit + Receipt Cards 🔴

1. **RabbitMQ প্রশ্নের নিষ্পত্তি (৩০ মিনিট):** RabbitMQ কোথায় চলত? Notification/SOP সার্ভিসের আলাদা রিপো আছে কি? সত্যিটা `decisions.md`-এ লেখো। **আসল না হলে** CV-র লাইন বদলাও (FN-এর কাছে CV আগেই গেছে, তাই ইন্টারভিউতে নিজে থেকে ঠিক করে দেওয়ার লাইনও রেডি রাখো: "On the CV I wrote RabbitMQ. To be precise, our broker today is Kafka; …")
2. **Node প্রশ্নের নিষ্পত্তি:** কোন প্রজেক্টে, কোন কোম্পানিতে, কী বানিয়েছিলে, প্রোডাকশনে গিয়েছিল কি না

3. **৬টা Receipt Card** (টেমপ্লেট নিচে), প্রতিটা CV সংখ্যার জন্য একটা: ২০% (সার্ভার লোড), ৬৫% (resolution time), ৮৫% (রিপোর্ট), ৩০% (engagement), ২৭% (রেভিনিউ), ৯৯.৯% (আপটাইম)

```
### Receipt: <সংখ্যা> — <দাবি>
- মেট্রিক: (p50 latency? CPU%? MTTR? DAU?)
- আগে → পরে: X → Y   · সময়কাল: কবে থেকে কবে মাপা
- কোথা থেকে: (Grafana / CloudWatch / Telescope / Jira টিকেটের সময় / অনুমান)
- আমার অংশ: আমি নিজে ___ করেছি; টিম ___ করেছে
- নিরাপদ বাক্য (EN): "…"
- পদ্ধতি না জানলে: "It was roughly X based on <source>. I'd want to re-measure before quoting it precisely."
```

> নিয়ম: যে সংখ্যার পদ্ধতি বলতে পারবে না, সেটা নিজে থেকে বলবে না। জিজ্ঞেস করলে "roughly" + সোর্স বলো।

D-7 / D-6 — ৬টা Anchor Project, প্রতিটা Depth Ladder দিয়ে

প্রতিটার জন্য Q-bank (FN-27) §A-তে L1–L5 স্কেলেটন আছে। নিজের ভাষায় ভরাট করো, তারপর **রেকর্ড করে** জোরে বলো।

#	Anchor	কেন এটা	স্টোরি ID	FN JD মিল
A1	Multi-tenant architecture + cache-scope gotcha	সবচেয়ে গভীর, কোডে যাচাইযোগ্য	S3	SaaS, MySQL
A2	Report / benchmark performance (fan-out, concurrency, TTL, rollups)	সংখ্যা আছে (১৮×), আসল root-cause গল্প	S2	MySQL, perf
A3	Observability: structured logging + SLO + retry UX	JD-তে সরাসরি SLI/SLO	S4	Observability
A4	Event-driven: Kafka observers (+ RabbitMQ যদি আসল হয়)	FN-এর microservices দিক	S1	Event-driven
A5	Task Manager ডিজাইন (MySQL vs Mongo, hierarchy, lock)	"Design judgment"-এর পরিষ্কার উদাহরণ	S9	Work-order-এর মতো approval flow
A6	AI-assisted engineering + QA Test Lab (journal / revert / guard)	নতুন (২০২৬-১০), safety-first চিন্তা দেখায়	S8	Team productivity

D-6 অতিরিক্ত: Sokrio-র আর্কিটেকচার খালি কাগজে ২ মিনিটে আঁকা প্র্যাকটিস করো (clients → subdomain → middleware → Laravel → MySQL landlord/tenant + Mongo + Redis + Kafka; FastAPI read-only)। Panel প্রায়ই বলে "can you sketch it?"। সোর্স: `/.claude/_architecture/system-overview.md`।

D-5 — "Why X not Y" কার্ড

Q-bank §B-এর ১০টা প্রশ্ন। প্রতিটার উত্তরের ছাঁচ:

> **Context** (আমাদের constraint কী ছিল) → **Options** (২–৩টা) → **Choice + why** → **Cost** (কী ছেড়ে দিলাম) → **Revisit** (কোন অবস্থায় সিদ্ধান্ত বদলাতাম)

শেষের "Revisit" অংশটাই senior-লেভেলের চিন্তা দেখায়। এটা কখনো বাদ দেবে না।

D-4 — Incidents, Debugging, Ownership

- **S4 incident ভরাট:** একটা আসল incident: কী ভাঙল → কীভাবে জানলে (alert? client?) → প্রথম ১৫ মিনিটে কী করলে → root cause → fix → আবার যেন না হয় সেজন্য কী করলে
- **S11 mistake:** ০৮-১৩-এর `ReportTooLargeException` revert কি এটা? না হলে অন্য একটা। ফরম্যাট: "I made X mistake → impact → how I owned it → the system change so it can't recur"
- **S10 disagreement:** একটা আসল ঘটনা (যেমন ডিজাইন সিদ্ধান্তে: Task Manager-এ Mongo বনাম MySQL?)
- **Debugging walkthrough ছাঁচ** (Q-bank §C): Reproduce → Scope (কোন টেন্যান্ট/কবে থেকে/কী বদলেছে) → Hypothesis → Measure (logs/EXPLAIN/APM) → Fix → Prevent

D-3 — Gaps + Field Nation ফিট

Honest Bridge ফর্মুলা (Q-bank §D), Node / NestJS / Kubernetes / Kafka প্রতিটার জন্য:

> "**Honestly**, my production depth is in X. **What transfers**: Y (concept-level, concrete). **What I've done to close it**: Z (fn-lite repo / course / reading Kafka layer). **First 30 days at FN** I'd ..."

FN-এর সমস্যায় নিজেকে বসাও (ম্যানেজার জিজ্ঞেস করবে "How would you approach our PHP→Node migration?"): `03-system-design/02-php-to-node-migration.md` রিভিশন। কিন্তু উত্তরে **তোমার Sokrio অভিজ্ঞতা** দিয়ে শুরু করো (Laravel + FastAPI পাশাপাশি একই DB-তে = দুই-স্ট্যাক সহাবস্থানের আসল অভিজ্ঞতা)।

D-2 — Live Mock HM-1 (৬০ মিনিট)

এই চ্যাটে Claude-কে দাও:

> "Act as a US-based Engineering Manager at Field Nation with one senior engineer on the panel. Interview me for 60 minutes on technical depth, not trivia. Pick 2 projects from my CV and drill each to L5 (what → how → why → what broke → what would you change). Challenge every number and every 'we'. Include one 'gap' question (Node/K8s/Kafka) and one 'how would you approach our problem' question. One question at a time. After the mock: score Depth / Ownership / Trade-offs / Honesty / Clarity (1–5) and list my top 5 fixes."

D-1 — Fix + Mock HM-2 + প্রশ্ন

- D-2-এর top 5 fix প্র্যাকটিস
- Mock HM-2: ৩০ মিনিট, শুধু L4–L5 আর "push-back" প্রশ্ন
- **ম্যানেজারকে করার ৫টা প্রশ্ন** (Q-bank §F থেকে বাছো)

D-0

Receipt Cards একবার পড়ো, pitch একবার জোরে বলো, ক্যামেরা/মাইক/নেট চেক। নতুন কিছু পড়বে না।

---

৪. শেষের মানদণ্ড (D-1-এ চেক)

- [ ] §১-এর সব 🔴 সারি resolved, CV-র কোনো লাইন নিয়ে ভয় নেই
- [ ] ৬টা Receipt Card: প্রতিটা সংখ্যার পদ্ধতি **অথবা** নিরাপদ "roughly" বাক্য
- [ ] A1–A6 প্রতিটা L5 পর্যন্ত, "I" vs "we" পরিষ্কার, প্রতিটা L1 ৩০ সেকেন্ডে
- [ ] ১০টা "Why X not Y" কার্ডের প্রতিটায় "Revisit" অংশ আছে
- [ ] S4, S10, S11 পুরো ভরাট (কোনো `[FILL]` নেই)
- [ ] ৪টা Honest Bridge উত্তর জোরে বলা
- [ ] Mock HM-1-এ প্রতিটা ডাইমেনশনে ≥ ৪/৫

> Tracker ID: FN-27 · Generated: 2026-10-14 · Source: Sokrio কোডবেস (`.claude/_architecture/system-overview.md`, `_modules/*/memory.md`, sokrio-server git log), `_memory/story-bank.md`, FN SE job post প্ল্যান: FN-26 (`00-plan/02-fieldnation-hm-depth-plan.md`)

Hiring Manager রাউন্ড: Technical Depth Q-Bank

কীভাবে ব্যবহার করবে: প্রতিটা প্রশ্নে আগে **নিজে জোরে উত্তর দাও (রেকর্ড করো)**, তারপর এখানকার স্কেলেটনের সাথে মেলাও। `[Komol পূরণ করবে]` মানে কোড বা CV-তে তথ্যটা নেই। বানাবে না, নিজের মনে থাকা সত্যিটা লেখো।

প্রতিটা উত্তরের ছাঁচ: BLUF বাক্য (EN) → ২–৩টা কংক্রিট ডিটেইল → "my part" → থেমে জিজ্ঞেস করো "Want me to go deeper?"

ইন্টারভিউয়ার কী মাপছে (৫টা ডাইমেনশন):

ডাইমেনশন	ভালো সিগন্যাল	খারাপ সিগন্যাল
Depth	L4–L5 পর্যন্ত নামতে পারে, ফাইল/টেবিল/কনফিগ লেভেলের ডিটেইল	শুধু buzzword: "we used microservices for scalability"
Ownership	"I designed / I decided / I was on call" + টিমকে ক্রেডিট	সব "we", বা সব "I"
Trade-offs	বিকল্প + খরচ + কখন বদলাবে	"X is best practice"
Honesty	"I don't know, here's how I'd find out"	বানানো সংখ্যা, ধরা পড়লে ঘুরিয়ে বলা
Clarity	BLUF, ৯০ সেকেন্ড, ডায়াগ্রাম আঁকতে পারে	৫ মিনিটের মনোলগ

---

§A — Anchor Projects: Depth Ladder (L1 → L5)

A1. Multi-tenant Architecture

L1 What (EN, ৩০ সেকেন্ড):

> "Sokrio is a multi-tenant SaaS: one landlord MySQL database for platform data, and one MySQL database **plus** one MongoDB database per client. Every request arrives on the client's subdomain, and middleware switches both connections before any of our code runs."

L2 How:

- `RouteServiceProvider` সব রুটকে `Route::domain("{account}.$appHost")`-এ মোড়ায়, তাই subdomain প্রতিটা রুটের প্যারামিটার
- `EnsureTenantAccess`: `LOWER(domain)` দিয়ে Org খোঁজে (৭ দিন cache `org_by_domain_{domain}`) → inactive হলে 403 → `TenantManager::tenant($org)->reconnect()`
- `reconnect()` একসাথে tenant MySQL **আর** MongoDB সুইচ করে, কারণ দুটোর db নাম একই (`org.domain`)

- তারপর `auth:sanctum` → `permission:` gate। Migration দুই ভাগে: `base:migrate` (landlord) আর `tenant:migrate {db?}`
- FastAPI benchmark সার্ভিস `X-Tenant-Domain` header দিয়ে টেন্যান্ট রিজলভ করে, শুধু read করে

L3 Why (DB-per-tenant কেন, shared-schema + `tenant_id` কেন না):

- Enterprise ক্লায়েন্টের জন্য isolation সবচেয়ে জোরালো: ভুল query-ও অন্য টেন্যান্টের ডেটা ছোঁয় না
- প্রতি টেন্যান্টের backup/restore/migration আলাদা করা যায় (যেমন একটা টেন্যান্টের dump লোকালে নেওয়া)
- বড় টেন্যান্ট শব্দ করলেও (noisy neighbour) অন্যদের index/লক প্রভাবিত হয় না
- **খরচ:** N টা DB-তে migration চালাতে হয়, cross-tenant analytics কঠিন (তাই আলাদা FastAPI fan-out সার্ভিস), connection management জটিল
- `[Komol পূরণ করবে: সিদ্ধান্তটা কি তুমি নিয়েছিলে, নাকি তুমি আসার আগেই ছিল? (তোমার প্রথম commit ২০২৩-০৮-২৮)। CV "built from scratch" বলছে। সত্যিটা বলো: "I extended / hardened it" নাকি "I designed it"]`

L4 Break (এখানেই জেতা যায়):

- **Cache scope gotcha:** `ScopeCacheToTenant` শুধু HTTP request-এ cache prefix দেয়। Queue job-এ সেই prefix থাকে না, তাই job request-এর লেখা key দেখতে পায় না (আর উল্টোটা ঘটলে টেন্যান্টের ডেটা মিশে যাওয়ার ঝুঁকি থাকে)। নিয়ম: প্রতিটা cache key-তে টেন্যান্ট/টেরিটরি স্কোপ নিজে বসাও, middleware-এর ওপর ভরসা করবে না
- **Connection switch ভুলে যাওয়া:** landlord-এ ফিরে না গেলে পরের query ভুল DB-তে যায়। নিয়ম: switch করার পর সবসময় `DB::setDefaultConnection(CONN_LANDLORD)`
- **স্কেল ১০×:** ১০০০+ DB হলে migration-এর সময়, connection pool, প্রতি DB-র overhead বাড়বে। তখন ছোট টেন্যান্টদের shared-schema-তে নেওয়ার (hybrid) কথা ভাবা যায়
- `[Komol পূরণ করবে: কোনো near-miss হয়েছিল? যেমন ভুল টেন্যান্টের cache দেখানো?]`

L5 Reflect (EN):

> "If I redid it, I'd make tenant context explicit in jobs, passed as a parameter and asserted, rather than relying on middleware state, and I'd add an automated test that fails if any cache key lacks a tenant prefix."

🔗 Sokrio link: S3 · কোড: `EnsureTenantAccess`, `TenantManager`, `ScopeCacheToTenant`

A2. Report / Benchmark Performance

L1 (EN):

> "Our cross-tenant market-benchmark endpoint was slow because it fanned out over every tenant's MongoDB. I traced the cost to that fan-out, then fixed it in layers: bounded adaptive concurrency, shorter default lookback windows, much longer cache TTLs on the expensive path, missing indexes, and finally monthly pre-aggregated rollups."

L2 How:

- Root cause: `/category-benchmark`-এর all-tenant fan-out (`gather_tenants`) ছিল সবচেয়ে বড় খরচ
- **Adaptive concurrency:** `_adaptive_concurrency(n_orgs)`, `[8, 24]`-এ বাঁধা, ~প্রতি ৫ টেন্যান্টে ১ স্লট। ফিক্সড মানের বদলে টেন্যান্ট সংখ্যার সাথে বাড়ে, যাতে ১০০+ টেন্যান্টে ৩০ সেকেন্ডের wall budget কম ব্যবহার না হয়
- Partial result হলে সেটা জানানো হয়, চুপচাপ ভুল সংখ্যা দেখানো হয় না
- **Lookback:** ফিক্সড ৩ মাস → enum (১/২/৩/৬, ডিফল্ট ১), ৬-এর বেশি হলে `400 INVALID_DATE_RANGE`, org lookup-এর **আগে** চেক
- **TTL:** category cache 600s → 10800s = **১৮× কম recompute**
- **Indexes:** boot-এ index তৈরি করা হয় না (সিদ্ধান্ত), আলাদা `python -m app.db.indexes` + `--check` (সব active টেন্যান্টে gap শূন্য)
- **Rollups:** `order_rollups_monthly` per-tenant, scheduled job লেখে
- Laravel দিকে: stock details report-এ pagination + background export, outlet report-এ memory limit

L3 Why:

- Rollup কেন **scheduled**, event-driven না: `order_summaries` লেখে Laravel, তাই event-driven হলে Laravel-এ hook লাগত, দুই সার্ভিস কাপল হয়ে যেত। Benchmark ডেটা কয়েক ঘণ্টা পুরনো হলেও চলে
- Concurrency কেন বাঁধা: কম হলে ধীর; বেশি হলে Mongo-তে চাপ আর অন্য টেন্যান্টের live traffic-এ প্রভাব
- TTL কেন এত লম্বা: মাসিক benchmark ডেটা ঘণ্টায় ঘণ্টায় বদলায় না। ৩ ঘণ্টা staleness ব্যবসায়িকভাবে চলে
- **"Sequential chains left untouched":** `_find_org → MySQL mappings → tenant_outlet_ids` ধাপগুলো একটা আরেকটার ওপর নির্ভরশীল, তাই parallel করা যায় না

L4 Break:

- Cache stampede: TTL একসাথে শেষ হলে সব request একসাথে recompute করবে। প্রতিকার: lock / stale-while-revalidate / jitter `[Komol পূরণ করবে: আমাদের কোনটা আছে?]`
- একটা টেন্যান্ট ধীর হলে পুরো fan-out আটকায়। তাই per-tenant timeout আর partial result
- Rollup job ব্যর্থ হলে পুরনো ডেটা দেখায়। মনিটরিং কী? `[Komol পূরণ করবে]`
- **Before/after সংখ্যা:** `[Komol পূরণ করবে: p50/p95 আগে → পরে, কোথা থেকে মাপা]`

L5 (EN):

> "The biggest lesson: measure before tuning. Our first instinct was caching, but the real cost was the fan-out shape. Next I'd move the fan-out entirely off the request path. Rollups do that for the monthly view; I'd extend the same idea to the other benchmark tabs."

🔗 Sokrio link: S2 · মডিউল: `market-benchmark-optimization`, `market-benchmark-rollups`

A3. Observability

L1 (EN):

> "For the benchmark service I added structured JSON-lines logging. Every request records tenant, duration, cache hit, error code and request id, so a slow success and a slow failure can be

> compared side by side. On the frontend, each section fails independently with a retry, instead of the whole page failing."

L2 How:

- `log_event()` → `logs/{module}.jsonl`, ফিল্ড: `{tenant, level, duration_ms, cache_hit, error_code, message, request_id}`
- Bare `Exception`-এর জন্য global handler, path prefix দেখে module ঠিক হয় (`/api/v1/market-benchmark` → `market_benchmark`)
- Frontend: `SectionError` (মেসেজ + Retry, দ্বিতীয়বার ব্যর্থ হলে "reported to the team"), প্রতিটা সেকশনে skeleton
- CV: Grafana/Prometheus, CloudWatch, Telescope, SLO-ভিত্তিক alert, ৬টা অ্যাপের on-call

L3 Why:

- স্ট্রাকচার্ড লগ grep/aggregate করা যায়, তাই "কোন টেন্যান্ট ধীর?" এক লাইনের query
- SLO-তে alert, raw CPU-তে না: ইউজার যা অনুভব করে সেটাই মাপো, alert fatigue কমে

L4 Break:

- ফাইলে লগ = একটা সার্ভারেই সীমাবদ্ধ, rotation লাগে। স্কেল হলে centralized pipeline (CloudWatch/Loki/ELK) লাগবে
- Request ID কি Laravel → FastAPI পর্যন্ত যায় (distributed tracing)? `[Komol পূরণ করবে]`

L5: "Next step is shipping those JSON lines to a central store and defining an SLO on p95 duration per tenant, with an error-budget alert."

SLI/SLO-র সংজ্ঞা (মুখে বলতে পারতে হবে): SLI = মাপা জিনিস (যেমন p95 latency < 2s হওয়া request-এর %) · SLO = টার্গেট (৯৯.৯%) · Error budget = ০.১% = ৩০ দিনে ~৪৩ মিনিট

🔗 Sokrio link: S4 · মডিউল: `market-benchmark-error-logging`, `geo-benchmark-observability`

A4. Event-driven (Kafka, আর RabbitMQ যদি আসল হয়)

> ⚠️ **আগে Truth Audit (FN-26 §১)।** কোডে Kafka আছে, RabbitMQ নেই। Kafka commit-এর author টিমমেট। নিচের স্কেলেটন কোড থেকে নেওয়া; তোমার অংশ সৎভাবে বসাও।

L1 (EN, সৎ ভার্সন):

> "Domain events like order, delivery, department changes and GPS tracking are published to Kafka from Eloquent observers, and consumers update downstream read models. A teammate built the Kafka layer; my part was `[Komol পূরণ করবে: e.g. consuming events in X, debugging Y, reviewing Z]`."

L2 How (কোড থেকে):

- Contracts: `MessageProducerInterface`, `MessageConsumerInterface`, `ConsumerFactoryInterface`, `TopicRegistryInterface`, `TopicProvisionerInterface`, `KafkaMessageHandlerInterface`
- Publish: `OrderObserver`, `DeliveryObserver`, `DepartmentChangeObserver` → `PublishesKafkaMessage` trait; Tracking actions (`WriteTrackingActions`, `WriteBulkTrackingActions`)
- Consumer: `DepartmentMessageHandler`, dynamic group ID + error retry, max runtime 6h, consumer group isolation fix, টেস্টে publish স্কিপ

L3 Why — জানতেই হবে এমন trade-off:

- Kafka vs RabbitMQ: Kafka = log, replay করা যায়, partition-এর ভেতরে ordering, উচ্চ throughput (GPS tracking-এর জন্য মানানসই) · RabbitMQ = smart routing (exchange), per-message ack, task queue-এর জন্য সহজ
- Observer থেকে publish করার ঝুঁকি: DB commit হলো কিন্তু publish ব্যর্থ, বা উল্টোটা (dual-write problem) → **Transactional Outbox** হলো সঠিক সমাধান
- At-least-once delivery → consumer idempotent হতে হবে (event ID দিয়ে dedupe)

L4 Break:

- "Consumer group isolation fix" কেন লেগেছিল? (দুই environment/টেন্যান্ট একই group ID শেয়ার করলে একে অপরের মেসেজ খেয়ে ফেলে) `[Komol পূরণ করবে: আসল কারণ]`
- Consumer পিছিয়ে পড়লে (lag) কী হয়? কীভাবে মনিটর করো?
- Poison message → retry → DLQ

L5: "I'd add an outbox table so publishing is part of the same DB transaction, and consumer-lag alerts tied to an SLO."

🔗 Sokrio link: S1 · কোড: `app/Contracts/Kafka/*`, `app/Observers/Concerns/PublishesKafkaMessage.php`

A5. Task Manager: Design Judgment

L1 (EN):

> "Task Manager lets supervisors plan tasks for field staff, approve them, and lock achievements. Similar in shape to a work-order lifecycle. The interesting part was a few design calls made against existing precedent."

L2/L3 সিদ্ধান্ত + কারণ:

- **MongoDB-র বদলে MySQL**, যদিও Claim/Complaint-এর মতো আগের মডিউল Mongo-তে ছিল। কারণ: task ডেটা কম ভলিউমের আর `users` / `departments` / `territories`-এর সাথে ভারী join লাগে, তাই relational FK ভালো
- **নতুন `user_supervisors` টেবিল:** আগে পুরো কোডবেস audit করে নিশ্চিত হওয়া যে কোনো `supervisor_id`/reporting line নেই; একমাত্র কাছাকাছি জিনিস Mongo-র `TeamAttendance`, যেটা point-in-time snapshot, source of truth না
- **One-way lock** (`achievement_locked_at`), Phase 1-এ admin reopen নেই: scope ছোট রাখা, অডিটযোগ্যতা

- Route sequencing scope-এর বাইরে, কারণ schema-তে জায়গা ছিল না আর story list-এও ছিল না
- সব টেন্যান্টে ডিফল্ট চালু (`org_modules` gate নেই)

L4: lock ভুল হলে? → পরের ধাপে audited unlock endpoint · supervisor বদলালে ইতিহাস? → `user_supervisors`-এ effective date লাগবে কি না `[Komol পূরণ করবে]`

L5 (EN): "The habit I'm proudest of is auditing before adding a table. Duplicated hierarchies are a common source of bugs."

🔗 Sokrio link: S9 · FN মিল: work-order approval flow · মডিউল: `task-manager`

A6. AI-assisted Engineering + QA Test Lab

L1 (EN):

> "I set up our AI-assisted workflow: named agents for DB, backend, frontend and tests, with project rules and a phase-by-phase process where every diff gets reviewed. Most recently I built a QA scenario seeder that loads known-answer test data into a tenant, verifies reports against expected results, and reverts cleanly."

L2 How (QA Test Lab):

- `qa:seed <module> <tenant> --synthetic --apply` → `qa:verify` → `qa:revert`
- **Write-ahead journal** (Mongo `qa_seed_runs` + `qa_seed_journal`): প্রতিটা write-এর আগে `pending` এন্ট্রি, পুরো `before` row সহ → `applied` / `failed` → `reverted`
- Revert উল্টো ক্রমে চলে, conflict পেলে থামে (`--force` ছাড়া)
- **Safety guard:** শুধু local host-এ write; staging host deny-list-এ; `APP_ENV=production` প্রত্যাখ্যাত; MySQL আর Mongo দুই host-ই প্রিন্ট করে কনফার্ম করে

L3 Why: আগে টেস্ট ডেটা হাতে বানাতে হতো, আর রিপোর্টের "সঠিক উত্তর" জানা থাকত না। Journal কারণ: revert নিশ্চিত না হলে QA-কে আসল টেন্যান্টে চালানো নিরাপদ না

- AI নিয়ে গার্ডরেইল: প্রতিটা diff নিজে review, টেস্ট বাধ্যতামূলক phase, ভুল সংশোধন হলে "lesson" হিসেবে রেকর্ড যাতে আবার না হয়

L4: AI ভুল কোড দিলে কীভাবে ধরো? `[Komol পূরণ করবে: একটা আসল উদাহরণ]` · Journal নিজেই ব্যর্থ হলে?

L5: "AI made me faster at the mechanical parts, so I spend the saved time on verification. The Test Lab exists because faster code needs stronger proof."

🔗 Sokrio link: S8 · মডিউল: `qa-scenario-seeder`

---

§B — "Why X not Y" কার্ড (D-5)

ছাঁচ: **Context → Options → Choice + why → Cost → Revisit**

#	প্রশ্ন (EN)	উত্তরের কেন্দ্র	🔗
B1	"Why database-per-tenant instead of a `tenant_id` column?"	Isolation, per-tenant backup/restore, noisy neighbour · খরচ: N migrations, cross-tenant analytics · Revisit: হাজারো ছোট টেন্যান্ট হলে hybrid	A1
B2	"Why both MySQL and MongoDB? Isn't that extra complexity?"	MySQL = relational/transactional (orders, users); Mongo = পূর্ব-aggregated summary, flexible document · খরচ: দুই DB-র ops, consistency · Revisit: Task Manager-এ ইচ্ছা করে MySQL বেছে নেওয়া (A5) দেখায় যে প্রতিটা কেসে বিচার করো	A2, A5
B3	"Why a separate Python service for benchmarks?"	Read-only, cross-tenant fan-out, async I/O; Laravel request cycle-কে চাপমুক্ত রাখা · খরচ: দুই স্ট্যাক, duplicated tenant resolution	A2
B4	"Why Kafka?" (বা RabbitMQ, সত্যি হলে)	GPS tracking-এর ভলিউম, replay, একাধিক consumer · খরচ: ops ভারী · Revisit: ছোট task queue-র জন্য Redis queue যথেষ্ট (Laravel-এ সেটাই আছে)	A4
B5	"Scheduled rollups vs event-driven updates?"	Laravel-এর সাথে কাপলিং এড়ানো, staleness চলে · Revisit: real-time dashboard লাগলে	A2
B6	"Cache: how do you decide TTL and invalidation?"	ডেটা কত দ্রুত বদলায় × recompute কত দামি · tenant-scoped key · stampede	A2, A1
B7	"Monolith vs microservices: would you split again?"	ভাগ করো যখন deploy/স্কেল/টিম আলাদা হয়, ফ্যাশনের জন্য না · Sokrio-র core এখনো Laravel মনোলিথ + আলাদা সার্ভিস = pragmatic	A4
B8	"Offset vs cursor pagination?"	বড় টেবিলে offset ধীর আর পেজের মাঝে ডেটা বদলালে row বাদ পড়ে বা দুবার আসে; cursor স্থির · Sokrio-তে `paginate()` (offset), রিপোর্টে background export	—
B9	"Sync API call vs queue job for heavy reports?"	৩০ সেকেন্ডের বেশি লাগলে → queue + background export + notify · retry/DLQ	A2
B10	"REST vs GraphQL for your mobile app?"	Versioned REST (`mob`, `v2`, `v3`) = cache-যোগ্য, সহজ; GraphQL over-fetch কমায় কিন্তু caching/অনুমতি জটিল	—

---

§C — Incidents, Debugging, Ownership (D-4)

C1. "Tell me about a production incident you owned." → S4 ভরাট করো:

> Detect (কীভাবে জানলে, কত মিনিটে) → Mitigate (প্রথমে রক্তপাত থামানো: rollback/feature flag/cache clear) → Root cause → Fix → **Prevent** (alert/test/runbook) → Communicate (ক্লায়েন্ট/টিমকে কী জানালে)

`[Komol পূরণ করবে: একটা আসল incident। নিচের কোনোটা কি মনে পড়ে? stock report-এ memory/timeout, Kafka consumer থেমে যাওয়া, cache-এ ভুল টেন্যান্ট, migration ব্যর্থ]`

C2. "Tell me about a mistake you made." → S11

- প্রার্থী ঘটনা: ২০২৬-০৮-১৩ `ReportTooLargeException` + stock-details-report পরিবর্তন → ০৮-১৫ revert। `[Komol পূরণ করবে: কেন revert হলো? কী শিখলে? এর পরের "Limit memory…" commit কি আসল fix`

`ছিল?]`

- ছাঁচ (EN): "I shipped X. It caused Y. I reverted within Z, told the team, found that ***, and since then I always*** ."

C3. "Walk me through how you'd debug: a report is slow for one client only."

> "First I scope it: one tenant or all, since when, what changed (deploy, data volume). Then measure, not guess: request logs with duration and cache_hit, then EXPLAIN on the main query in that tenant's DB, since per-tenant DBs can have different data shapes and index states. Common culprits: a missing index on that tenant (we have an index-check script for exactly this), N+1 queries, or a cold cache. Fix, then add a check so it can't silently regress."

🔗 Sokrio link: A1 (per-tenant DB), A2 (`--check` index script), A3 (logs)

C4. "Describe a disagreement." → S10 `[Komol পূরণ করবে]`। প্রার্থী: Task Manager-এ MySQL বনাম Mongo, যেখানে আগের precedent ছিল Mongo **C5. "What's the hardest bug you've fixed?"** `[Komol পূরণ করবে]`। Cache-scope (HTTP বনাম job) ধরনের বাগ সবচেয়ে ভালো, কারণ এটা অদৃশ্য আর L4-লেভেলের **C6. "How do you know your code works before it ships?"** → feature test + QA Test Lab (`qa:verify` = known-answer) + staging + phase-by-phase review **C7. "Tell me about a time you pushed back on product/scope."** → Task Manager scope-এর বাইরে রাখা (route sequencing, reopen action) `[তোমার ভূমিকা কী ছিল?]`

---

§D — Gaps: Honest Bridge (D-3)

ছাঁচ: **Honestly → What transfers → What I've done → First 30 days**

D1. Node.js / NestJS (EN):

> "Honestly, my production depth is PHP/Laravel and Python/FastAPI. `[Node কোথায় ব্যবহার করেছ: Komol পূরণ করবে]` . What transfers directly: NestJS is structurally close to Laravel, with modules, DI container, controllers, DTO validation, guards like middleware. Async I/O I use daily in FastAPI (asyncio fan-out with bounded concurrency). To close the gap I built `fn-lite` , a small NestJS work-orders service with TypeORM and MySQL. In the first 30 days I'd pair on an existing Node service and ship a small change end-to-end."

D2. Kubernetes:

> "I've deployed with Docker and docker-compose and managed Linux VPSes, but I haven't operated a K8s cluster myself. I understand the model: Deployments, Services, Ingress, readiness vs liveness probes, HPA. I'd expect to be a K8s *user* first, writing manifests and reading pod logs, and I'd learn your deploy pipeline in week one."

D3. Kafka: (Truth Audit-এর পর)

> "Our platform publishes domain events to Kafka. A teammate built that layer, and I `[work with it how]`. I understand the core trade-offs: partitions and ordering, consumer groups, at-least-once delivery so consumers must be idempotent, and the dual-write problem that an outbox solves."

D4. React Native: "I've contributed screens and fixes; my React depth is web. RN shares the component and state model; what's new is navigation, native modules, offline storage and release builds."

> ❌ কখনো বলবে না: "I can learn anything quickly" (প্রমাণ ছাড়া)। ✅ বলবে: একটা কংক্রিট প্রমাণ যে আগে দ্রুত শিখেছ (যেমন Vue → React/TS রিরাইট ২ মাসে, S6)

---

§E — "How would you approach OUR problem?" (FN-specific)

প্রশ্ন (EN)	তোমার Sokrio অ্যাঙ্কর
"We're moving PHP to Node microservices. How would you do it safely?"	Strangler fig, একটা bounded context দিয়ে শুরু, shared DB আগে → পরে আলাদা। **আসল অভিজ্ঞতা:** Laravel আর FastAPI একই DB-তে পাশাপাশি চলে, tenant resolution দুই জায়গায়। এই duplication-এর সমস্যাটাও বলো · দেখো `03-system-design/02-php-to-node-migration.md`
"A technician's check-in shows the wrong location. How do you investigate?"	GPS-spoofing detection (S9) + tracking pipeline (Kafka tracking actions)
"Clients complain the work-order list is slow at peak."	A2 + C3 ছাঁচ
"How would you define SLOs for our dispatch API?"	A3: SLI choice, error budget, alert on burn rate
"How do you work with a US team 10–11 hours away?"	1–10 PM শিফট ঠিক আছে (D7); async written updates, ADR/ডক, PR description-এ কারণ লেখা; S7

---

§F — Hiring Manager-কে করার প্রশ্ন (৫টা বাছো)

1. "What does a great first 90 days look like for this role?"
2. "Where are you in the PHP → Node transition, and what's the hardest part right now?"
3. "How do you handle on-call and SLOs across the Dhaka and US teams?"
4. "How are technical decisions made, for example who writes the spec and how is disagreement resolved?"
5. "What's the biggest source of production incidents today?"
6. "How do you measure engineering success on this team?"
7. "What would make someone in this role *not* succeed?"

---

§G — Panel-এর push-back লাইন (দ্রুত রিফ্লেক্স)

ওরা বলে	তুমি বলো (EN)
"How did you measure that?"	Receipt Card থেকে। না জানলে: "Roughly X, based on Y. I'd want to re-measure before quoting it precisely."
"Was that you or the team?"	"The team owned ***; I personally*** ."
"Isn't that over-engineered?"	"Fair challenge. The constraint that justified it was ***. If __ changed, I'd simplify to*** ."
"What would you do differently?"	একটা আসল জিনিস (L5), "nothing" কখনো না
"I don't think that's right."	"Let me think. You may be right about ***; my reasoning was*** . How do you handle it here?"
একটা টেকনোলজি যা জানো না	"I haven't used it in production. Here's how I'd reason about it from ___."